

Инструкция по деплою проекта Tour Uzbekistan на Ubuntu

Frontend + Admin + Backend NestJS + Prisma + PostgreSQL. Документ описывает production-развертывание, проверку, обновления, логи и типовые ошибки.

1. Архитектура проекта

В репозитории три самостоятельные части, которые на сервере должны работать вместе:

Часть	Папка	Роль	Production-режим
Основной сайт	front	Публичный Vue/Vite сайт для пользователей	Собирается в front/dist и отдается Nginx
Админка	admin	Vue/Vite панель управления контентом	Собирается в admin/dist и отдается Nginx
Backend	backend	NestJS API, Prisma, JWT, Swagger, загрузки медиа	Запускается как Node.js процесс через PM2
База данных	PostgreSQL	Хранение пользователей, контента, туров, заявок, медиа и логов	Запускается в Docker Compose или как системный PostgreSQL

Рекомендуемая схема доменов для production:

- <https://example.com> - основной сайт.
- <https://admin.example.com> - админка.
- <https://api.example.com> - backend API, Swagger и файлы из /uploads.

Такой вариант проще всего обслуживать: фронт и админка имеют разные домены, а CORS на backend явно разрешает оба origin.

2. Требования к серверу

- Ubuntu 22.04 LTS или 24.04 LTS.
- Доступ по SSH под пользователем с sudo.
- Открытые порты 80 и 443 для Nginx.
- Node.js версии 20.19+ или 22.12+; проект Vite требует именно такую версию.
- npm, Git, Docker Compose, Nginx, Certbot, PM2.
- Минимум 2 GB RAM, лучше 4 GB RAM, особенно при сборке frontend/admin на сервере.

```
sudo apt update
sudo apt install -y git curl nginx ca-certificates ufw
```

```
# Docker
sudo apt install -y docker.io docker-compose-plugin
sudo systemctl enable --now docker
```

```
# Node.js 22 LTS через NodeSource
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt install -y nodejs
node -v
npm -v
```

```
# PM2
```

```
sudo npm install -g pm2
pm2 -v
```

Если команда `node -v` показывает версию ниже 20.19, сборка Vite может падать. Обновите Node перед установкой зависимостей.

3. Подготовка проекта на сервере

Создайте директорию приложения и загрузите код. В примерах используется путь `/var/www/tour-uzbekistan`.

```
sudo mkdir -p /var/www/tour-uzbekistan
sudo chown -R $USER:$USER /var/www/tour-uzbekistan
cd /var/www/tour-uzbekistan

# Клонирование проекта из GitHub
git clone https://github.com/Sovyshkin/tour-uzbekistan.git .

# Если проект загружается архивом:
# распакуйте архив так, чтобы внутри были папки backend, front и admin
```

После загрузки проверьте структуру:

```
ls -la
ls -la backend front admin
```

4. Настройка PostgreSQL

В проекте уже есть `backend/docker-compose.yml`. Для production можно запустить только PostgreSQL из compose, а backend держать под PM2. Это удобно: база изолирована, а Node-процесс легко обновлять.

4.1. Безопасная правка docker-compose

В production лучше не публиковать PostgreSQL наружу на все интерфейсы. В файле `backend/docker-compose.yml` измените порт базы так:

```
ports:
  - "127.0.0.1:5432:5432"
```

Также замените пароль `postgres` на сильный пароль. Такой же пароль понадобится в `DATABASE_URL`.

4.2. Запуск базы

```
cd /var/www/tour-uzbekistan/backend
docker compose up -d postgres
docker compose ps
```

Если порт 5432 занят, проверьте, кто его использует:

```
sudo ss -ltnp | grep 5432
docker ps
```

5. Настройка backend .env

Создайте production env-файл в `backend/.env`. Значения секретов ниже являются примерами, их нужно заменить.

```
cd /var/www/tour-uzbekistan/backend
cp .env.example .env
nano .env

NODE_ENV=production
PORT=3000
DATABASE_URL=postgresql://postgres:<DB_PASSWORD>@localhost:5432/
tour_uzbekistan?schema=public
JWT_SECRET=<LONG_RANDOM_ACCESS_SECRET>
JWT_EXPIRES_IN=1d
JWT_REFRESH_SECRET=<LONG_RANDOM_REFRESH_SECRET>
JWT_REFRESH_EXPIRES_IN=7d
CORS_ORIGIN=https://example.com,https://admin.example.com
```

- DATABASE_URL должен указывать на PostgreSQL. Если backend запускается не в Docker, а через PM2, host обычно localhost.
- CORS_ORIGIN должен содержать домен основного сайта и домен админки через запятую.
- JWT_SECRET и JWT_REFRESH_SECRET должны быть разными длинными строками. Не используйте change-me.

Сгенерировать секреты можно так:

```
openssl rand -hex 32
```

6. Установка и запуск backend

Установите зависимости, примените миграции Prisma, при необходимости заполните базу seed-данными, затем соберите проект.

```
cd /var/www/tour-uzbekistan/backend
npm ci
npm run prisma:generate
npx prisma migrate deploy
```

```
# Seed выполняйте только если это первая установка и нужна тестовая/стартовая база.
npm run db:seed
```

```
npm run build
```

Папка загрузок должна существовать и быть доступна пользователю, от которого работает PM2:

```
mkdir -p uploads/media
chmod -R 755 uploads
```

Запуск через PM2:

```
cd /var/www/tour-uzbekistan/backend
pm2 start npm --name tour-uzbekistan-backend -- run start
pm2 save
pm2 startup
```

Команда pm2 startup выведет строку с sudo. Скопируйте и выполните ее, чтобы backend автоматически запускался после перезагрузки сервера.

6.1. Проверка backend

```
curl -i http://127.0.0.1:3000/api/v1/health
curl -I http://127.0.0.1:3000/api/docs
```

После seed-добавления тестовые доступы для проверки админки:

- superadmin@centrum-holidays.test / admin12345
- manager@centrum-holidays.test / manager12345

7. Сборка основного сайта

Перед сборкой основного сайта задайте production API URL. В текущем frontend коде VITE_API_BASE_URL должен быть абсолютным URL, потому что API helper использует JavaScript URL API.

```
cd /var/www/tour-uzbekistan/front
npm ci
nano .env.production

VITE_API_BASE_URL=https://api.example.com/api/v1

npm run build
```

После сборки Nginx будет отдавать статические файлы из /var/www/tour-uzbekistan/front/dist.

8. Сборка админки

Админка тоже должна знать адрес backend API. Для нее используется такой же env-параметр.

```
cd /var/www/tour-uzbekistan/admin
npm ci
nano .env.production

VITE_API_BASE_URL=https://api.example.com/api/v1

npm run build
```

После сборки Nginx будет отдавать статические файлы из /var/www/tour-uzbekistan/admin/dist.

9. Настройка Nginx

Ниже пример конфигурации с тремя доменами: основной сайт, админка и API. Замените домены на реальные.

```
sudo nano /etc/nginx/sites-available/tour-uzbekistan

server {
    listen 80;
    server_name example.com www.example.com;

    root /var/www/tour-uzbekistan/front/dist;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }
}

server {
    listen 80;
    server_name admin.example.com;

    root /var/www/tour-uzbekistan/admin/dist;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }
}

server {
    listen 80;
    server_name api.example.com;

    client_max_body_size 50m;

    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

```

    }
}

sudo ln -s /etc/nginx/sites-available/tour-uzbekistan /etc/nginx/sites-enabled/tour-uzbekistan
sudo nginx -t
sudo systemctl reload nginx

```

Если на сервере включен default-site и он мешает доменам, отключите его:

```

sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t
sudo systemctl reload nginx

```

10. SSL-сертификаты

После того как DNS доменов указывает на сервер, выпустите HTTPS-сертификаты через Certbot.

```

sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d example.com -d www.example.com -d admin.example.com -d api.example.com
sudo certbot renew --dry-run

```

После включения HTTPS убедитесь, что в backend/.env переменная CORS_ORIGIN содержит именно https-домены, а не http.

11. Firewall

Откройте только SSH, HTTP и HTTPS. PostgreSQL и backend-порт 3000 должны быть доступны только локально.

```

sudo ufw allow OpenSSH
sudo ufw allow 'Nginx Full'
sudo ufw enable
sudo ufw status

```

12. Финальная проверка после деплоя

- 1 Откройте <https://api.example.com/api/v1/health>. Должен вернуться JSON health-check.
- 2 Откройте <https://api.example.com/api/docs>. Должен открыться Swagger.
- 3 Откройте <https://example.com>. Главная страница должна загрузить данные из backend.
- 4 Откройте <https://admin.example.com>. Должна открыться форма входа в админку.
- 5 Войдите в админку, откройте страницы, страны, туры, услуги, новости, медиа, заявки и бронирования.
- 6 Загрузите тестовое изображение в медиа и проверьте, что оно доступно через URL вида <https://api.example.com/uploads/...>
- 7 Измените тестовый текст в админке и проверьте, что основной сайт показывает новое значение.

13. Обновление проекта на сервере

Типовой порядок обновления после новых изменений в GitHub-репозитории <https://github.com/Sovyshkin/tour-uzbekistan>:

```

cd /var/www/tour-uzbekistan
git pull

cd backend
npm ci
npm run prisma:generate
npm run prisma:migrate:deploy
npm run build
pm2 restart tour-uzbekistan-backend

cd ../front
npm ci

```

```

npm run build

cd ../admin
npm ci
npm run build

sudo nginx -t
sudo systemctl reload nginx

```

Если были изменения только во frontend/admin, backend можно не перезапускать. Если были изменения Prisma schema или backend src, обязательно выполните migrate deploy и перезапустите PM2.

14. Логи и диагностика

Что проверить	Команда
Backend runtime logs	pm2 logs tour-uzbekistan-backend
Статус backend	pm2 status
PostgreSQL Docker logs	docker logs tour-uzbekistan-postgres
Nginx ошибки	sudo tail -f /var/log/nginx/error.log
Nginx access	sudo tail -f /var/log/nginx/access.log
Порты	sudo ss -ltnp
Миграции Prisma	cd backend && npx prisma migrate status

15. Резервные копии

Минимально нужно сохранять PostgreSQL и папку backend/uploads. Пример ручного backup:

```

mkdir -p /var/backups/tour-uzbekistan

docker exec tour-uzbekistan-postgres pg_dump -U postgres tour_uzbekistan \
> /var/backups/tour-uzbekistan/db-$(date +%F-%H%M).sql

tar -czf /var/backups/tour-uzbekistan/uploads-$(date +%F-%H%M).tar.gz \
-C /var/www/tour-uzbekistan/backend uploads

```

Восстановление базы из SQL backup:

```

cat /var/backups/tour-uzbekistan/db-YYYY-MM-DD-HHMM.sql | \
docker exec -i tour-uzbekistan-postgres psql -U postgres tour_uzbekistan

```

16. Частые проблемы

CORS ошибка в браузере

- Проверьте CORS_ORIGIN в backend/.env.
- Там должны быть точные origin фронта и админки: протокол + домен, без слеша в конце.
- После правки env выполните pm2 restart tour-uzbekistan-backend.

502 Bad Gateway на api.example.com

- Проверьте, жив ли backend: pm2 status и curl http://127.0.0.1:3000/api/v1/health.
- Проверьте Nginx proxy_pass на 127.0.0.1:3000.

Админка открывается, но login не работает

- Проверьте admin/.env.production: VITE_API_BASE_URL должен вести на production API.
- Пересоберите админку после изменения env: npm run build.

- Проверьте Swagger endpoint `/api/v1/admin/auth/login`.

Изображения не отображаются

- Проверьте, что backend раздает `/uploads`.
- Проверьте права на `backend/uploads`.
- Проверьте, что frontend/admin используют API URL с правильным origin, потому что медиа строятся от origin backend.

Prisma migrate deploy падает

- Проверьте `DATABASE_URL` и доступность PostgreSQL.
- Проверьте статус миграций: `npx prisma migrate status`.
- Не используйте `migrate dev` на production. Для сервера нужна команда `npx prisma migrate deploy`.

17. Production checklist

- Node.js версии 20.19+ или 22.12+ установлен.
- PostgreSQL запущен и не открыт наружу без необходимости.
- `backend/.env` заполнен production-секретами.
- `CORS_ORIGIN` содержит домены сайта и админки.
- Prisma migrations применены через `migrate deploy`.
- Backend запущен через PM2 и добавлен в автозагрузку.
- `front/.env.production` и `admin/.env.production` указывают на production API.
- `front/dist` и `admin/dist` собраны.
- Nginx настроен с `try_files` для Vue Router.
- HTTPS включен для всех доменов.
- Проверены health, Swagger, сайт, админка, медиа, заявки и бронирования.
- Настроены регулярные backup базы и uploads.